

Bloc: A Distributed Bouldering Session Tracker with Dual-Client Architecture

Alfie James Jenkins

Falmouth University – COMP204 Distributed Systems

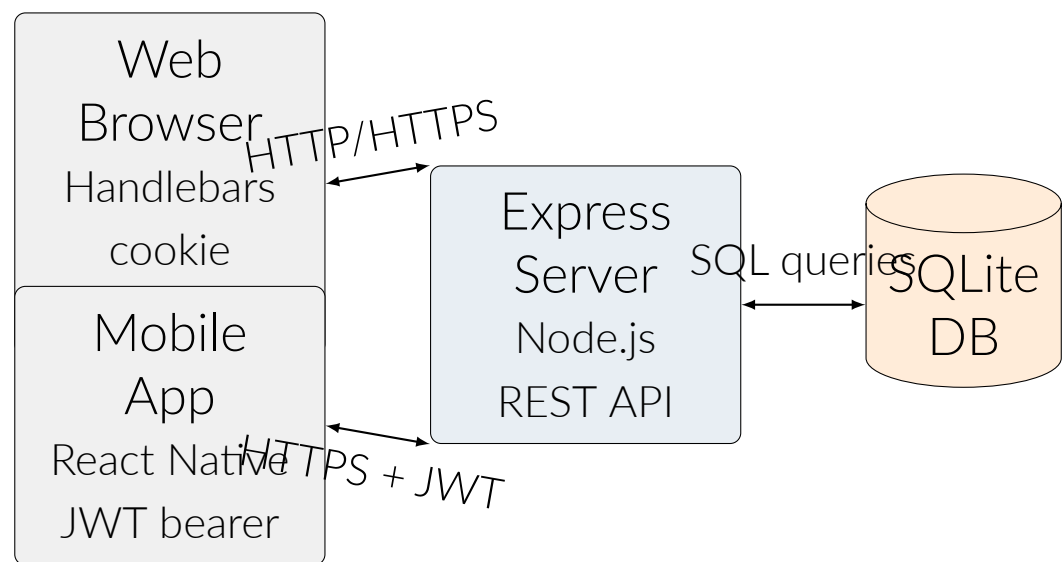
The Bloc System

Bloc tracks bouldering sessions. Climbers log gym visits and individual climbs from a browser or a phone; all data goes to the same backend, so both views stay in sync.

The project is built around one distributed systems pattern: **multiple independent clients sharing a common server and data store**. Neither client holds state locally — both read and write through the same REST API, so a session started on the web shows up immediately in the mobile app.

Hosted on: a Linux VPS running Node.js

System Architecture



Browsers handle cookies automatically, so the web app uses cookie-based sessions. The mobile app has no shared cookie jar — it sends a JWT in every **Authorization: Bearer** header instead. Both clients hit the same database rows.

Technology Stack

Layer	Technology
Mobile client	React Native, Expo, TypeScript
Web client	Express, Handlebars, CSS
REST API	Express (Node.js)
Database	SQLite3
Auth (web)	bcrypt + express-session
Auth (mobile)	bcrypt + JWT (jsonwebtoken)
File uploads	Multer (5 MB limit, image validation)
Hosting	Linux VPS (Node.js, port 3000)

Core Features

- **Timed sessions** — start a session at a named gym; **end_time** is stamped when the user finishes
- **Climb logging** — grade (V-scale), attempts, topped status, zones reached, optional photo per climb
- **Photo uploads** — **multipart/form-data** via Multer; stored in **public/uploads/** and served as static files; nullable in the schema
- **Public feed** — all completed sessions visible to every user; lets climbers see what others at the gym have logged
- **Shared state** — web and mobile read the same rows; no synchronisation layer required
- **Single active session** — enforced at query level (**end_time IS NULL**), consistent across both clients

Future Development

- **Grade progression charts** — visualise topped percentage per grade over time using session history already in the database
- **Object storage for images** — replace **public/uploads/** with S3-compatible storage to decouple files from the server process
- **PostgreSQL migration** — **db.js** is the only module to change; the rest of the stack is database-agnostic
- **Push notifications** — alert a user when a friend logs a session at the same gym, using the public feed as the data source
- **Offline logging** — queue climbs locally when the device has no signal and sync when connectivity returns

References

- Fielding, R. T. (2000). *Architectural Styles and the Design of Network-based Software Architectures*. University of California, Irvine.
- Jones, M. B., Bradley, J., & Sakimura, N. (2015). *JSON Web Token (JWT)*. RFC 7519. IETF.
- Provos, N., & Mazieres, D. (1999). *A Future-Adaptable Password Scheme*. USENIX Annual Technical Conference.
- Express.js documentation: expressjs.com
- Expo documentation: docs.expo.dev
- React Native documentation: reactnative.dev
- Multer documentation: github.com/expressjs/multer

REST API Design

The mobile API has eight endpoints. All session and climb routes go through **requireApiAuth**, which verifies the JWT and attaches **req.userId** before the route handler runs.

Method	Endpoint	Auth
POST	/api/auth/login	Public
POST	/api/auth/register	Public
GET	/api/feed	Public
GET	/api/sessions	JWT
POST	/api/sessions	JWT
GET	/api/sessions/:id	JWT
POST	/api/sessions/:id/end	JWT
POST	/api/sessions/:id/climbs	JWT

GET /api/feed returns the 20 most recent completed sessions with climb counts, using a **LEFT JOIN** so sessions with no climbs still appear.

The Mobile Auth Challenge

Cookie sessions work for browsers because the browser attaches the cookie to every same-origin request automatically. Mobile HTTP clients have no shared cookie jar — each request arrives with no identity attached.

Sending the session ID manually would mimic cookie behaviour, but without the browser's built-in protections. A JWT with its own expiry is the right call for a stateless HTTP client.

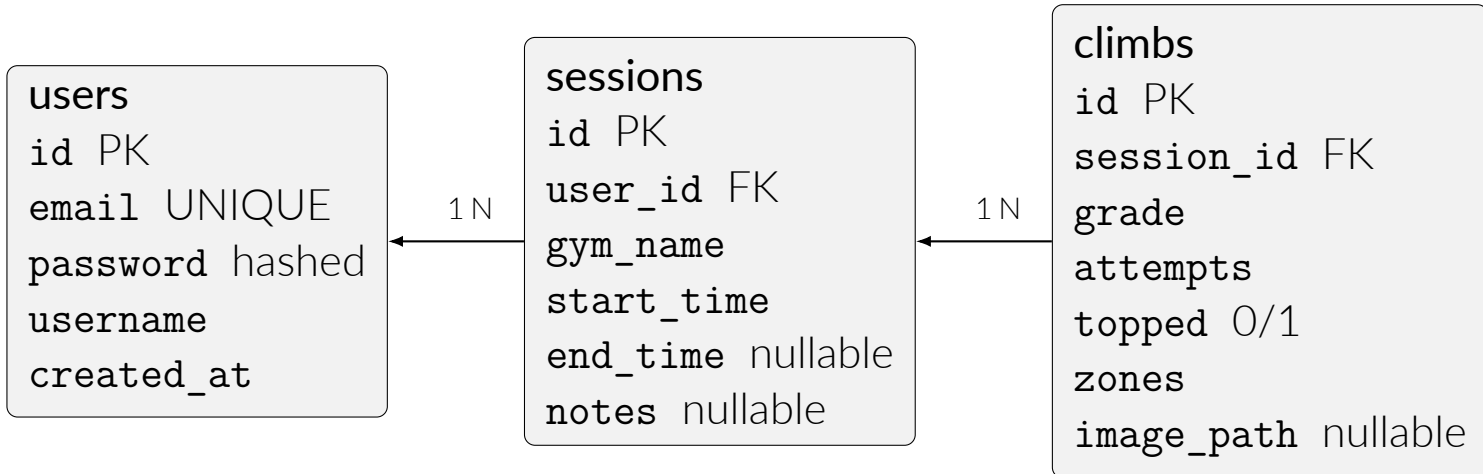
Solution: JWT Authentication Middleware

```
1 const requireApiAuth = (req, res, next) => {
2   const authHeader = req.headers['authorization'];
3   // Header format: "Bearer <token>"
4   const token = authHeader && authHeader.split(' ')[1];
5
6   if (!token)
7     return res.status(401).json({ error: 'Auth_Required' });
8
9   jwt.verify(token, JWT_SECRET, (err, decoded) => {
10    if (err)
11      return res.status(401).json({ error: 'Invalid_token' });
12    req.userId = decoded.userId; // injected for route handlers
13    next();
14  });
15 };
```

Tokens are signed with **HS256** and expire after 7 days. The app keeps the token in React Context and attaches it to every request; the JWT encodes the user identity directly, so no session state is needed on the server.

Database Schema

Three tables with cascading foreign keys. All state lives in SQLite; neither client caches data locally.



end_time IS NULL means the session is still running. Duration is calculated from **end_time - start_time** at query time and never stored, so editing a session cannot produce stale totals.

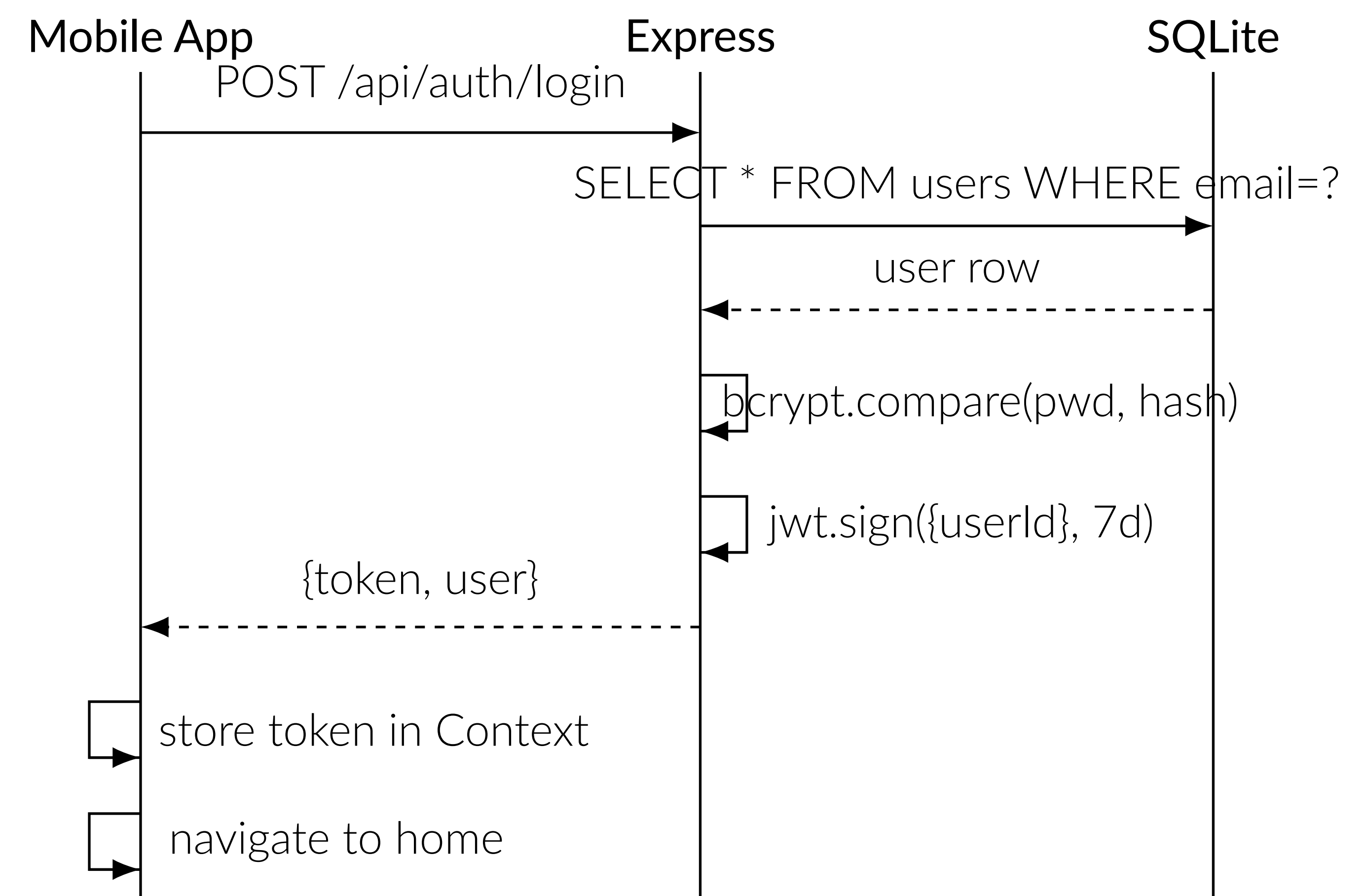
Web Client vs Mobile Client

Property	Web Browser	Mobile App
Technology	Express + Handlebars	React Native / Expo
Auth mechanism	Cookie session	JWT Bearer token
Auth middleware	requireAuth	requireApiAuth
Image upload	HTML form (POST)	FormData (fetch)

Both clients share the same database — the difference is in how they authenticate, not what they store.

Authentication Sequence

Mobile login flow, from tap to authenticated state:



Key Design Decisions

Session-first workflow

You have to start a session before logging a climb. One gym visit, one session; climbs cannot exist without one.

One active session per user

Before inserting a new session the server checks for an existing row where **end_time IS NULL**. If one exists, its ID comes back in the response so the client goes straight to it.

Dual auth

Browsers get a cookie session handled automatically; mobile clients get a signed JWT stored in React Context and sent as **Authorization: Bearer**. Same **users** table, same bcrypt hashes — only the transport differs.

Log-Climb Sequence (Distributed Flow)

Adding a climb crosses three service boundaries in one action: JWT auth, file storage, and a database write.

